

The Ultimate Guide to UI Architectures; MVC, MVP, MVVM

Before we look at the specific patterns, let's understand *why* they exist.

When you build an app screen (like the PlateMate Feed), you have three basic things:

1. **The Data:** The list of food items from your database.
2. **The Logic:** What happens when a user clicks "Claim".
3. **The Screen:** The actual buttons, text, and colors the user sees.

If you throw all three of these into one massive file, your code becomes an unreadable, untestable nightmare. UI Architecture Patterns (like MVC, MVP, and MVVM) are simply **rules for how to separate the Data, the Logic, and the Screen.**

1. MVC (Model-View-Controller)

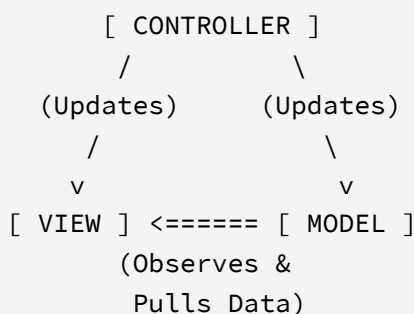
The Traditional Standard

MVC was the original way to build user interfaces. It separates concerns, but leaves a direct connection between the Screen and the Data.

The Roles

- **The Model (Data):** Handles the database and business logic.
- **The View (Screen):** The UI elements. In MVC, the View is "smart" – it knows how to observe the Model and update itself.
- **The Controller (Logic):** Listens to user clicks and tells the Model to update.

The Visualization (Triangle)



The Analogy: Fast Food Counter

1. You (User) tell the Cashier (Controller) you want a burger.
2. The Cashier tells the Kitchen (Model) to make it.
3. The Kitchen puts the finished burger on the counter.

4. You (View) see the burger on the counter and grab it yourself.

The Code Example (PlateMate)

```
// CONTROLLER: Hears the click, tells the Model.
function onClaimClicked(postId) {
    Model.claimFood(postId);
}

// MODEL: Updates the database.
function claimFood(postId) {
    database.update(postId, 'claimed');
    // Model tells the View that things changed
    View.refresh();
}

// VIEW: Pulls the new data directly from the Model.
function refresh() {
    let foodStatus = Model.getStatus(); // View talks directly to Model!
    if (foodStatus == 'claimed') {
        button.color = 'gray';
    }
}
```

The Problem: The View and the Model are tightly connected. In mobile apps, the Controller and View often merge into one giant, untestable file (famous in iOS development as the "Massive View Controller" problem).

2. MVP (Model-View-Presenter)

The Strict Middleman

MVP was created to fix MVC's biggest flaw. It completely cuts off the View from the Model. The View is made "dumb" and only does exactly what it's told.

The Roles

- **The Model (Data):** Same as MVC.
- **The View (Screen):** Completely dumb. It has zero logic. It only reports clicks to the Presenter and obeys commands.
- **The Presenter (Logic):** The absolute dictator. It gets data from the Model, formats it, and gives the View explicit UI commands.

The Visualization (Straight Line)

```
[ VIEW ] <=====> [ PRESENTER ] <=====> [ MODEL ]
  (Dumb UI)   (Commands) | (Asks)   (Data) |
                  v             v
                "Change button to gray" "Update database"
```

The Analogy: Fancy Restaurant

1. You (View) tell the Waiter (Presenter) your order.
2. The Waiter tells the Kitchen (Model).
3. The Kitchen gives the food to the Waiter.
4. The Waiter (Presenter) manually places the food on your table (View). You never look at or speak to the kitchen.

The Code Example (PlateMate)

```
// VIEW: Completely dumb. Just reports the click.
function onClick() {
  Presenter.handleClickClick(postId);
}

// PRESENTER: The brain. Handles everything and commands the View.
function handleClickClick(postId) {
  Model.claimFood(postId); // Tell model
  let newStatus = Model.getStatus(); // Get data back

  // Explicitly command the View
  if (newStatus == 'claimed') {
    View.makeButtonGray();
    View.showSuccessMessage("Food Claimed!");
  }
}
```

The Problem: The Presenter has to write hundreds of lines of code just micro-managing the View (`View.hideLoading()`, `View.showText()`, `View.changeColor()`). It gets exhausting.

3. MVVM (Model-View-ViewModel)

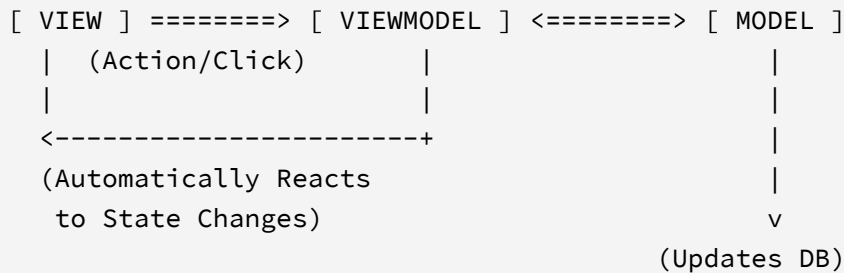
The Modern Standard (Used in React Native!)

MVVM takes the strict separation of MVP, but eliminates the exhausting micro-management using a magic trick called **Data Binding** (or "Reactivity").

The Roles

- **The Model (Data):** Same as always.
- **The ViewModel (State):** This holds "Variables of State" (e.g., `isLoading = true`, `buttonColor = 'gray'`). It updates these variables, but it **never** writes commands to the View. It doesn't even know the View exists.
- **The View (Screen):** The View secretly "listens" (binds) to the ViewModel's variables. When a variable changes, the View updates *itself* instantly.

The Visualization (One-Way Reactivity)



The Analogy: The Restaurant Pager

1. You order food, and the Kitchen gives you a digital buzzer (The View).
2. The Kitchen's computer system tracks orders (The ViewModel: `orderReady = false`).
3. When the food is done, the kitchen updates their computer: `orderReady = true`.
4. Your buzzer, which is wirelessly listening to that system, instantly lights up and vibrates. The kitchen didn't come tell you; your buzzer just reacted to the state change.

The Code Example (PlateMate in React Native)

Because React uses `useState`, it is naturally built for MVVM.

```
// VIEWMODEL: Just manages State. Doesn't know about UI elements.
function useClaimViewModel() {
  const [status, setStatus] = useState('active'); // State variable

  function handleClaim(postId) {
    Model.claimFood(postId);
    setStatus('claimed'); // Update state, and walk away.
  }

  return { status, handleClaim };
}

// VIEW: Binds to the ViewModel's state. Reacts automatically.
function FeedScreen() {
  // Connect to the ViewModel
  const { status, handleClaim } = useClaimViewModel();

  // The UI automatically changes based on the 'status' variable
  return (
    <Button
      color={status === 'claimed' ? 'gray' : 'blue'}
      text={status === 'claimed' ? 'Claimed!' : 'Claim Food'}
      onPress={() => handleClaim(12)}
    />
  );
}
```

4. The Cheat Sheet Comparison

Feature	MVC	MVP	MVVM	Who handles User Input?	Controller	Presenter
	View (passes to ViewModel)			Does View talk to Model?	YES (Directly reads data)	NO (Strictly forbidden)
	NO (Strictly forbidden)	NO (Strictly forbidden)		How does UI update?	View manually pulls from Model.	Presenter commands the View.
					View automatically reacts to ViewModel's state.	
				Best used for...	Older Web Frameworks (Ruby on Rails, older Spring).	Older Android apps (Java), Desktop Apps.
					Modern Mobile (React Native, iOS SwiftUI, Android Jetpack Compose).	

Summary for your PlateMate Team:

Because you are using React Native, **you must use MVVM**. React Native is declarative, meaning it's designed to react to changing variables (`useState`). If you try to use MVP and manually command buttons to change colors, you are fighting against the React framework!